SwarRakshak System Architecture

1. Architecture objective

SwarRakshak is a privacy-aware voice-risk assistant for detecting possible voice-cloning impersonation during sensitive calls. The hackathon architecture must be simple enough for a beginner team to build, but structured enough to show how the prototype could later become an enterprise security service.

The architecture should support this flow:

> **Capture audio → analyze short segments → compare available evidence → combine call context → calculate risk → explain the result → recommend verification.**

The prototype must not automatically approve, reject, or execute financial transactions. It should provide a warning and a recommended next action to a human user.

2. Recommended architecture style

Use a **modular monolith** for the hackathon.

A modular monolith is one application that contains clearly separated internal modules. It is easier to build and deploy than many independent microservices. The modules can later be separated into services if the product grows.

Why not use microservices immediately?

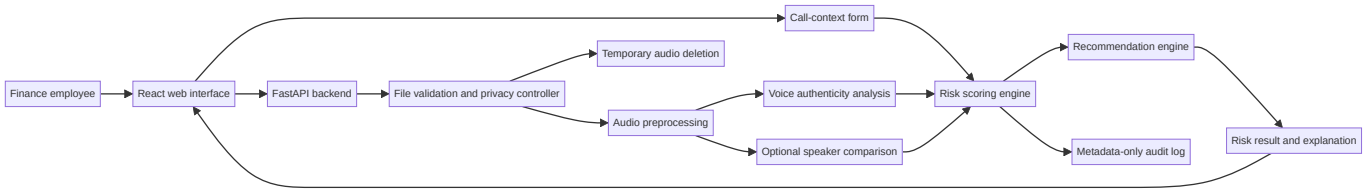
Microservices would introduce extra work for networking, authentication, deployment, monitoring, and debugging. The hackathon team should spend its time demonstrating the security workflow rather than managing infrastructure.

Recommended MVP modules

1. **Web interface module** for audio upload, context entry, and results.
2. **API module** for receiving requests and returning responses.
3. **Audio preprocessing module** for validation, conversion, and segmentation.
4. **Voice authenticity module** for synthetic-speech analysis.
5. **Speaker comparison module** for optional reference matching.
6. **Context and policy module** for structured risk factors.

7. **Risk scoring module** for combining signals.
8. **Recommendation module** for producing the next safe action.
9. **Privacy and audit module** for deleting audio and storing minimal metadata.

3. High-level architecture



4. Component responsibilities

4.1 Web interface

The web interface is the part the user sees. It should be intentionally simple.

Main screens

- **Home screen:** Explains the purpose of SwarRakshak and starts an analysis.
- **Audio input screen:** Uploads a current-call sample and optionally a reference sample.
- **Context screen:** Captures caller and transaction details.
- **Analysis screen:** Shows processing progress and segment status.
- **Result screen:** Shows risk level, score, confidence, reasons, and recommended action.
- **Incident summary screen:** Displays metadata that can be downloaded or copied.

UI requirements

- Do not use color as the only way to communicate risk.
- Display both a label and text, such as **High risk — pause and independently verify**.
- Explain that the result is a warning, not proof of identity.
- Show whether raw audio will be deleted after analysis.

4.2 API layer

The API layer coordinates the workflow. It should accept the audio, context fields, and optional reference sample, then return a structured result.

For the hackathon, the API can run in the same backend process as the other modules.

Responsibilities include:

- Validating request fields.
- Creating an analysis event ID.
- Passing audio to preprocessing.
- Calling the analysis modules.
- Calling the risk engine.
- Returning the result to the frontend.
- Handling errors without exposing sensitive implementation details.

4.3 File validation and privacy controller

This module runs before audio processing.

It should:

- Accept only supported audio formats.
- Enforce a maximum file size and duration.
- Generate a random temporary file name.
- Store files in a temporary directory outside the public web folder.
- Reject suspicious or malformed uploads.
- Mark whether a file is temporary or explicitly retained.
- Trigger deletion after analysis.

The prototype should use synthetic or consented audio only.

4.4 Audio preprocessing module

The preprocessing module prepares audio for the analysis models.

Typical steps are:

1. Convert the file to WAV format.
2. Convert it to a standard sample rate, such as 16 kHz, if required by the selected model.

3. Convert stereo audio to mono when appropriate.
4. Check duration and signal quality.
5. Divide the audio into short windows, for example two to five seconds.
6. Pass the windows to the analysis modules.

The module should return a clear error when the audio is too short, silent, corrupted, or excessively noisy.

4.5 Voice authenticity analysis module

This module estimates whether a segment contains patterns associated with synthetic or manipulated speech.

For the hackathon, it may use one of the following approaches:

- A suitable pretrained audio deepfake-detection model.
- A public audio-classification model adapted for the prototype.
- A baseline classifier using extracted acoustic features.
- Prepared demonstration results, clearly labeled as simulated if a model cannot be integrated reliably.

The module should return evidence, not an absolute verdict.

Example output


```
{
  "available": true,
  "synthetic_probability": 0.86,
  "confidence": 0.74,
  "reason_codes": ["SYNTHETIC_AUDIO_PATTERN"],
  "segments": [
    {"start_seconds": 0, "end_seconds": 4, "score": 0.81},
    {"start_seconds": 4, "end_seconds": 8, "score": 0.91}
  ]
}
```

4.6 Optional speaker comparison module

This module compares the current-call sample with a trusted reference sample when the user provides one.

It should produce a similarity value and a confidence value. It must not be used as a standalone identity proof because genuine voices can change due to illness, microphones, background noise, aging, or network compression.

If no reference sample is provided, the result should be marked as unavailable. Missing evidence must not automatically be treated as a speaker mismatch.

Example output

```
{
  "available": true,
  "similarity": 0.28,
  "confidence": 0.69,
  "reason_codes": ["REFERENCE_VOICE_MISMATCH"]
}
```

4.7 Context and policy module

The context module accepts structured information from the user. For the hackathon, manual form entry is safer and faster than attempting to understand every spoken sentence automatically.

Suggested fields:

```
{
  "caller_number_status": "unknown",
  "request_urgency": "urgent",
  "transaction_value": "high",
  "procedure_status": "bypass_requested",
  "caller_relationship": "executive"
}
```

The module converts these fields into reason codes and risk contributions.

4.8 Risk scoring engine

The risk engine combines available signals into a transparent score between 0 and 100.

A practical hackathon formula is:

```
risk_score = audio_points
           + speaker_points
           + caller_points
           + transaction_points
           + urgency_points
```

Suggested starting values:

Signal	Maximum contribution
Synthetic-voice indication	40
Speaker mismatch	30
Unknown caller number	10
High-value request	10
Urgent request or procedure bypass	10

The engine should cap the score at 100 and record which signals contributed to it.

Initial risk bands

Score	Level	Meaning
0–29	Low	No strong warning from the available evidence; continue normal procedures
30–59	Medium	Slow down, avoid sensitive disclosure, and perform additional verification
60–100	High	Pause the requested action and independently verify the caller

These values are prototype settings. They are not validated production thresholds.

Handling incomplete evidence

The engine must distinguish between:

- **Low evidence of risk:** The available evidence does not show a strong warning.
- **Insufficient evidence:** The audio is too poor or too short to analyze reliably.

Insufficient evidence should lead to a cautious recommendation such as independent verification, even if the numerical risk score is not high.

4.9 Recommendation engine

The recommendation engine translates the score and reason codes into clear instructions.

Examples:

- **Low:** “Continue normal verification procedures. Do not rely on voice alone for sensitive actions.”
- **Medium:** “Do not share confidential information yet. Ask a verification question and confirm through a trusted channel.”
- **High:** “Pause the transfer. Call the person using a number from the approved directory and require second-person approval.”
- **Inconclusive:** “The audio could not be analyzed reliably. Treat the request as unverified and use an independent verification method.”

4.10 Audit and privacy module

The audit module stores only the information needed to understand the analysis event.

Recommended stored fields:

- Event ID.
- Timestamp.
- User or session identifier, if authentication is implemented.
- Risk score and level.
- Confidence values.
- Reason codes.
- Selected context fields.

- Model version or rule version.
- Whether raw audio was deleted.

Do not store raw audio by default. If the prototype temporarily stores audio, it must delete it after the result is generated.

5. End-to-end data flow

Step 1: User starts an analysis

The finance employee opens the application and selects **Start new call analysis**.

Step 2: User provides audio

The user uploads a short current-call sample or records one through the browser. A trusted reference sample may be uploaded if available.

Step 3: Backend validates the request

The API checks the file type, size, duration, and request fields. It creates an event ID and stores the audio temporarily outside the public web directory.

Step 4: Audio is preprocessed

The backend converts the audio to the required format, checks basic quality, and splits the current sample into short segments.

Step 5: Analysis modules run

The voice authenticity module analyzes each segment. The speaker comparison module runs only if a reference sample is available.

Step 6: User supplies context

The frontend sends the structured call context to the backend. In a simulated live flow, this may happen before or during audio analysis.

Step 7: Risk is calculated

The risk engine combines audio, speaker, and context signals. It creates a score, level, confidence value, reason codes, and segment results.

Step 8: Recommendation is generated

The recommendation engine selects a verification action based on the risk level and reasons.

Step 9: Result is shown

The frontend displays the score, risk label, reasons, confidence, segment timeline, and recommended action.

Step 10: Temporary data is deleted

The backend deletes the raw audio unless the user explicitly selected an authorized retention option. The audit module stores the metadata-only summary.

6. API design

6.1 Create an analysis

`POST /api/v1/analyses`

Use `multipart/form-data` for the current audio, optional reference audio, and context fields.

Request fields

Field	Type	Required	Description
`current_audio`	File	Yes	Current call sample
`reference_audio`	File	No	Consented trusted sample
`caller_number_status`	String	Yes	`known`, `unknown`, or `hidden`
`request_urgency`	String	Yes	`normal` or `urgent`
`transaction_value`	String	Yes	`low`, `medium`, or `high`
`procedure_status`	String	Yes	`normal` or `bypass_requested`
`caller_relationship`	String	Yes	`customer`, `vendor`, `executive`, or `unknown`

6.2 Example response


```
{
  "event_id": "evt_01J9SWR123",
  "risk_score": 82,
  "risk_level": "high",
  "confidence": 0.78,
  "audio_analysis": {
    "available": true,
    "synthetic_probability": 0.86
  },
  "speaker_analysis": {
    "available": true,
    "similarity": 0.28
  },
  "reasons": [
    "Audio contains patterns associated with synthetic speech",
    "Current voice differs from the reference sample",
    "Caller is using an unknown number",
    "Request is urgent and involves a high-value transaction"
  ],
  "recommended_action": "Pause the transfer and call the person using an
  "raw_audio_retained": false,
  "model_version": "prototype-1"
}
```


6.3 Get an analysis result

`GET /api/v1/analyses/{event_id}`

Returns the metadata-only result. It should not return raw audio unless explicit retention is implemented and authorized.

6.4 Health check

`GET /api/v1/health`

Returns service status and model availability.

Example:

```
{
  "status": "ok",
  "audio_model": "ready",
  "speaker_model": "ready"
}
```

7. Suggested database design

Use SQLite for the hackathon. PostgreSQL can replace it later without changing the main data model.

`analysis_events`

Column	Type	Purpose
`event_id`	Text	Unique analysis identifier
`created_at`	Timestamp	Event creation time
`risk_score`	Integer	Score from 0 to 100
`risk_level`	Text	Low, Medium, High, or Inconclusive
`confidence`	Real	Overall confidence value
`recommended_action`	Text	Suggested next step
`raw_audio_retained`	Boolean	Whether audio was retained
`model_version`	Text	Model or rules version

`analysis_reasons`

Column	Type	Purpose
`id`	Integer	Record identifier
`event_id`	Text	Related event
`reason_code`	Text	Stable machine-readable reason
`message`	Text	Human-readable explanation
`contribution`	Integer	Score contribution

`analysis_segments`

Column	Type	Purpose
`id`	Integer	Segment identifier
`event_id`	Text	Related event
`start_seconds`	Real	Segment start
`end_seconds`	Real	Segment end
`synthetic_probability`	Real	Segment-level result
`confidence`	Real	Segment-level confidence

Do not store the audio bytes in the database for the MVP.

8. Technology recommendation

Beginner-friendly stack

- **Frontend:** React with simple CSS.
- **Backend:** Python and FastAPI.
- **Audio conversion:** FFmpeg.
- **Audio features:** librosa.

- **Machine learning:** PyTorch or a suitable pretrained audio model.
- **Speaker embeddings:** A compatible pretrained speaker-embedding model, only if integration is manageable.
- **Database:** SQLite.
- **Charts:** Recharts or a simple CSS timeline.
- **Deployment:** Docker or a single hosted application.

Why this stack?

Python has strong audio and machine-learning libraries. FastAPI makes it straightforward to create a REST API. React can provide a clean user experience, but the team can use plain HTML and JavaScript if React would slow development.

9. Deployment architecture

Hackathon deployment

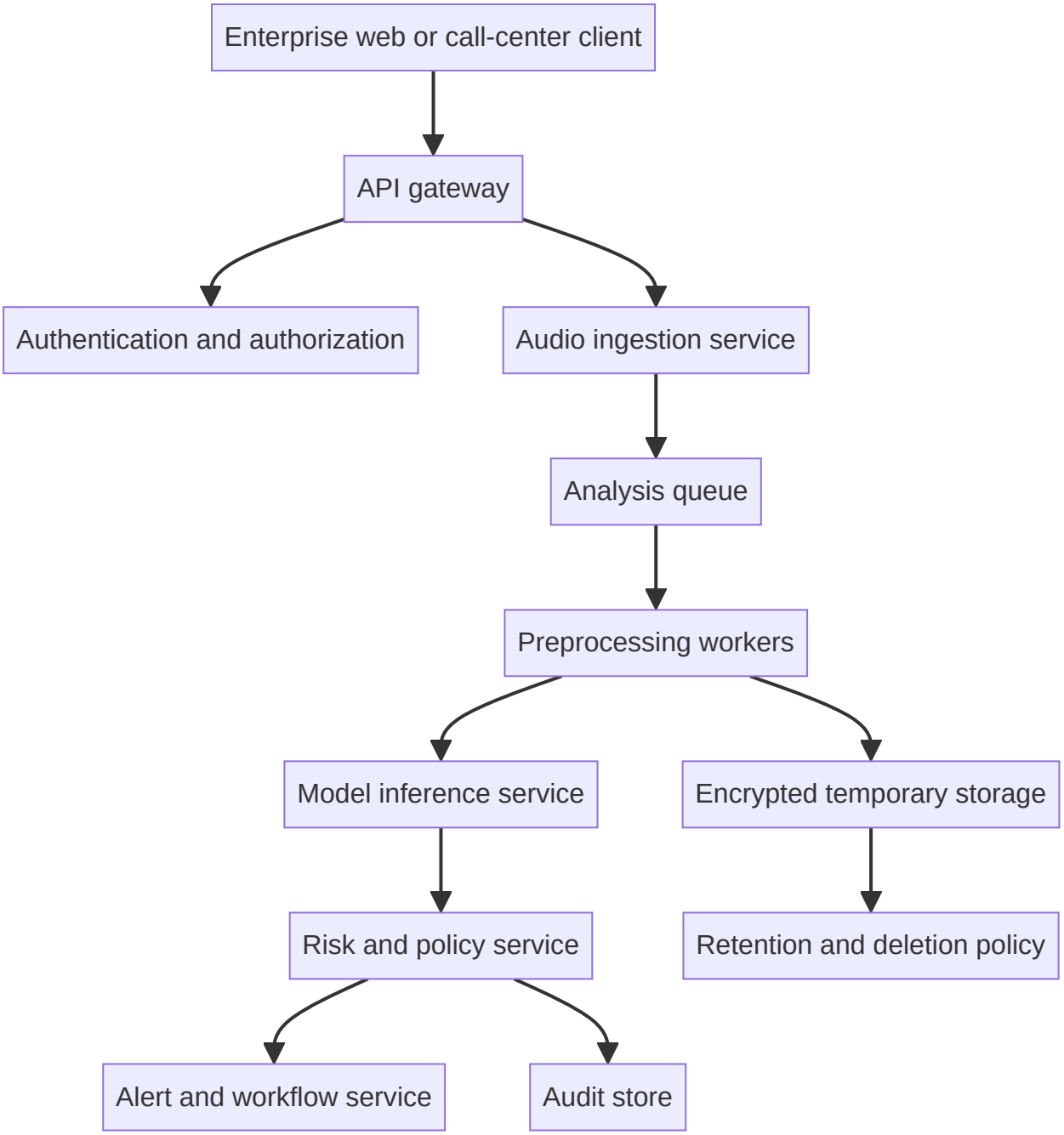
Use a single application deployment:

```
Browser
   |
   | HTTPS
   v
Frontend + FastAPI application
   |
   +-- Temporary local audio directory
   +-- SQLite database
   +-- Local or hosted inference model
```

This is sufficient for a demonstration.

Future production direction

A production version could separate the components:



The production design would require stronger identity management, encryption, observability, model monitoring, data-governance controls, and integration review.

10. Security and privacy design

Data minimization

Collect only the audio and context needed for the analysis. Do not collect unrelated personal information.

Temporary storage

Store incoming audio only in a temporary private location. Use random file names and delete files after processing.

Access control

For the MVP, a simple session may be sufficient. A production system should use organization-managed authentication, role-based permissions, and audit trails.

Transport security

Use HTTPS when the application is accessed over a network. Do not send sensitive audio through unencrypted HTTP in a deployed demo.

Input validation

Validate file extension, MIME type, file size, duration, and decoded audio format. Do not trust the file name supplied by the user.

Model and prompt safety

If transcription or a language model is added later, treat the transcript as untrusted input. Do not let spoken instructions directly execute transactions or change system settings.

Human approval

A High risk result should trigger a recommendation to verify. It should not automatically block a legitimate user or move money.

Privacy messaging

The interface should clearly state:

> “Audio is analyzed for this session and deleted by default. The result stores risk metadata and reason codes, not the recording.”

This statement must be changed if the implementation behaves differently.

11. Error handling

Situation	System behavior
Unsupported file format	Show supported formats and ask for a new file
File too large	Reject upload and show maximum size
Audio too short	Mark analysis as inconclusive and recommend independent verification
Silent or corrupted audio	Show a quality error and do not produce a confident score
Reference sample missing	Run audio and context analysis without speaker comparison
Model unavailable	Fall back to a labeled prototype rule or show unavailable status
Analysis timeout	Show a retry option and preserve no raw audio unless explicitly retained
Database error	Show a safe error; do not expose stack traces or file paths
Low confidence	Use cautious wording and emphasize independent verification

12. MVP versus future architecture

Area	Hackathon MVP	Future version
Audio source	Upload or browser recording	Telephony, VoIP, and collaboration integrations
Processing	Single backend process	Scalable worker and inference services
Model	Pretrained or prototype model	Continuously evaluated multilingual models
Speaker comparison	Optional reference upload	Consent-managed speaker profiles
Context	Manual form	Secure metadata and workflow integrations
Alerts	On-screen warning	Enterprise notifications and escalation workflows
Storage	SQLite metadata only	Encrypted event store with retention policies
Authentication	Basic session or demo login	Enterprise single sign-on and role-based access
Decision	Human follows recommendation	Configurable workflow with human approval gates
Monitoring	Basic logs	Model, latency, drift, and false-positive monitoring

13. Suggested development plan

Phase 1: Working interface

Build the upload form, context form, result page, and basic API endpoint. Use fixed sample responses first to prove the user journey.

Phase 2: Audio pipeline

Add file validation, FFmpeg conversion, segmentation, and temporary-file cleanup.

Phase 3: Analysis integration

Connect the selected audio model. Add speaker comparison only after the main voice-analysis path works.

Phase 4: Risk engine

Implement transparent scoring, risk bands, reason codes, and recommendations.

Phase 5: Demo scenarios

Prepare genuine, synthetic, and contextually suspicious samples. Confirm that each scenario produces the intended result.

Phase 6: Polish and safety

Add error states, privacy messaging, metadata-only summaries, accessible risk labels, and a clear explanation of model limitations.

14. Architecture decisions for the team

Decision 1: Use a modular monolith

This minimizes infrastructure and allows the team to focus on the product flow.

Decision 2: Analyze uploaded or browser-recorded audio

This demonstrates the concept without requiring telecom access.

Decision 3: Use manual context entry

This is more reliable for the MVP than trying to infer every transaction detail from conversation audio.

Decision 4: Use transparent scoring

A weighted rule engine is easy to explain to judges and easy to modify when the team learns more from test scenarios.

Decision 5: Treat speaker comparison as optional evidence

The product should still work when no trusted reference sample is available.

Decision 6: Delete raw audio by default

This demonstrates privacy by design and reduces unnecessary risk.

15. What the architecture intentionally does not promise

SwarRakshak does not promise to:

- Detect every AI-generated voice.
- Prove who is speaking.
- Replace multifactor authentication or callback verification.
- Work equally well for every language, accent, microphone, and network.
- Automatically make financial or legal decisions.
- Securely process real banking transactions in the hackathon prototype.

The correct product claim is narrower and safer:

> **SwarRakshak combines voice-analysis evidence and call context to identify risky situations and recommend independent verification before sensitive action is taken.**

16. Final recommended architecture

For the hackathon, implement the following minimum path:



This architecture is realistic for a beginner team, supports a convincing demo, and leaves a clear path toward future enterprise integrations.